HACKACHUSETTS 2026 / CIVIC ACTION TRACK

CodeLens

Revised engineering playbook v2: aligned to the official rules and built Kiro-first

Objective: Build a credible, demo-ready AI programming tutor that turns real Python execution into step-by-step, beginner-friendly lessons for students who do not have access to personalized instruction. This is a one-day execution plan, not a production system.

Definition of done by 7:30 PM (submissions lock at 8:00 PM)

A student opens a public URL, picks or edits a short Python program, clicks Visualize, steps forward and backward through accurate execution states, predicts the next value before it is revealed, and reads a plain-language explanation grounded in the real trace. Alongside it: a public GitHub repo with `/.kiro` committed at the root, a public demo video under three minutes, a written description, and a write-up of how Kiro was used.

Why there is a v2

The v1 plan is technically sound, but it was written as a generic hackathon plan. Checked against the official Hackachusetts rules, it leaves out the things that are actually scored or required. Kiro usage is a full judging criterion (and insufficient use can disqualify a project), and the submission requires a video, a public repo with the .kiro folder, and a Kiro write-up. Section 1 lists each gap and its fix. New and changed items are tagged **[NEW]** and **[CHANGED]** throughout.

1 Gap review: what v1 missed

Each row cites the rule or judging criterion it comes from. The top four are the ones that could cost the most points.

#	Shortcoming in v1	Why it matters	Fix in v2
1	Kiro is never mentioned. The plan describes hand-coding by four owners.	Implementation (Kiro usage) is one of three judging criteria. Rules say insufficient Kiro use can disqualify.	Section 9: Kiro-native workflow with specs, steering, hooks, MCP, property-based tests and Crew parallel waves, all committed in /.kiro.
2	Video is treated as a backup asset; demo is a 90-second live pitch.	A public video under 3 minutes is required, and judges may score from video and description alone.	Video becomes a first-class deliverable with a scripted 3-minute cut (Section 13), uploaded by 7:00 PM.
3	No mention of required artifacts: public repo, .kiro folder, testing link, Kiro write-up.	Missing items are grounds for an incomplete or disqualified submission.	Submission checklist mapped line-by-line to the rules (Section 14). Commit .kiro in the first commit.
4	No differentiation. Step-through Python visualizers already exist and are widely used.	Potential Value explicitly asks how saturated the market is.	Lead with what generic visualizers do not do: predict-then-reveal, trace-grounded explanations, error explainer, multilingual, screen-reader narration (Section 4).
5	Curated examples only; editing is optional.	A tutor that cannot run the student's own code is a weaker value story. Code runs in the student's own browser tab, so the risk is contained to that tab.	Editable code in the Pyodide worker with a step cap plus worker timeout. Curated-only remains the Tier 3 fallback.

#	Shortcoming in v1	Why it matters	Fix in v2
6	Two deployables (static UI plus FastAPI server) and no hosting plan for the LLM key.	Doubles deploy risk late in the day; judges need a working link.	One static site plus one serverless function for /api/explain. Deterministic explanations work with no backend at all.
7	Civic case is generic ("access is uneven").	Judges ask who benefits, how it scales, and about accessibility, equity and adoption.	Name concrete users, design for shared Chromebooks and slow Wi-Fi, map lessons to the Massachusetts DLCS framework.
8	Pyodide download and startup time not addressed.	A multi-second blank screen on venue Wi-Fi hurts the demo and first impressions.	Preload the worker on page open, show progress, render a precomputed trace instantly while Python loads.
9	Testing is a manual matrix only.	Rules ask specifically about property-based testing in Kiro.	Hypothesis property tests on the pure-Python tracer (Section 9.5).
10	Accessibility relies partly on color highlights.	Accessibility is named in Potential Value.	aria-live narration of every step, text change markers, full keyboard flow.
11	No credit budget; schedule assumes four people hand-coding.	Each participant has a fixed credit allowance; parallel agents burn credits fast.	Model plan per task type and a check of the usage meter after each wave (Section 15).

2 How the rules score you

The official rules list three criteria. Everything in this plan should map to at least one of them.

Criterion	What judges look for	CodeLens answer
Potential Value	Real, clearly defined problem; scalable; easy to use; accessible; equity and adoption; market saturation; who benefits and how it scales.	Students without tutors get step-level, grounded help in their own language, on any school Chromebook, with no account. Teachers share lessons as links. Differentiated from plain visualizers by active recall and grounded AI.
Implementation	Variety and depth of Kiro tools (IDE, CLI, Web, Crew) and features; a thoughtful strategy for AI development; integrations beyond the basics.	Spec per feature, steering files, test and contract hooks, GitHub and browser MCP, property-based tests on the tracer, Crew running parallel build waves. Evidence logged in KIRO_USAGE.md as you go.
Quality and Design	Creativity and originality; polished, intuitive UI; use of unique public resources; combining technologies in surprising ways.	Real CPython in the browser via WebAssembly plus an LLM that is only allowed to explain observed evidence. Lessons aligned to a public state curriculum framework. Clean three-pane UI.

Bonus to aim for: Best Use of Kiro goes to the highest Implementation score among the top 10% of projects, so a strong Kiro story is worth extra even if the track prize goes elsewhere.

3 Product scope and acceptance criteria

Ship this MVP

- Editor: syntax-highlighted Python with three tested lessons (loop, branch, function). **[CHANGED]** Editing is on by default, guarded by the step cap and worker timeout.
- Execution: deterministic sequence of trace snapshots from genuine Python execution; active-line highlight labelled “Next line to run”; Previous / Next.

- Visualization: variable table, array cards, before→after change markers; call depth when stable.
- **[NEW]** Predict-then-reveal: before a step that changes a variable, the student can guess the new value. Graded from the trace itself, so no LLM is needed.
- **[NEW]** Error explainer: when a program raises, show the failing line and a beginner explanation of the exception (NameError, TypeError, IndexError, ZeroDivisionError, SyntaxError first).
- Teaching: one- or two-sentence explanation per step; deterministic template always available, LLM version when the service is up.
- **[NEW]** Accessibility: aria-live narration of each step ("Step 5 of 18. Next line to run: line 4. total changed from 3 to 4."), keyboard-only flow, high contrast, no color-only signals.
- **[NEW]** Shareable lesson link: code and starting step encoded in the URL, so a teacher can share a lesson with no accounts or database.

Should-have if time allows

- **[NEW]** Explanation language toggle (English and Spanish first). The LLM handles translation; keep deterministic templates in both languages.
- **[NEW]** Guided lesson mode: each of the three lessons tagged with the Massachusetts Digital Literacy and Computer Science (DLCS) standard it supports. Verify the exact standard codes before citing them.
- Call-stack view for a simple recursive function (stretch only; stop at 4 PM if it is not working).

Acceptance tests

- For `numbers = [3, 1, 2]; total = 0; for n in numbers: total += n`, the student sees total go 0 → 3 → 4 → 6, can find the next line to run at every step, steps backward without re-running, and can predict 4 before it is revealed.
- `while True: pass` ends in a visible "step limit reached" state within two seconds and the editor stays usable.
- `print(x)` with x undefined ends with the failing line highlighted and a plain-language NameError explanation.
- With the explain endpoint switched off, every step still has a deterministic explanation.

Explicit non-goals

No user accounts, database, class roster, grading, multi-language execution, multiplayer, package installation, file system access, analytics, or server-side code execution.

4 Differentiation and civic story **[NEW]**

Step-through visualizers for Python are well established, and some already add AI help. Judges will know this. CodeLens should not pitch itself as "a visualizer with AI"; it should pitch the specific teaching behaviors that plain visualizers leave out.

Differentiator	Why it matters for learning	Cost today
Predict-then-reveal at each change	Active recall beats passive watching; turns a viewer into a tutor.	Low: answer is already in the next snapshot.
Explanations constrained to the trace	No invented values; every number shown came from real execution.	Medium: prompt plus validation.
Beginner error explainer	Tracebacks are the point where beginners without a tutor most often give up.	Low: templates per exception type.

Differentiator	Why it matters for learning	Cost today
Screen-reader step narration	Most visual tools are hard to use without sight; this is a real equity gain.	Low: one aria-live region.
Explanations in the student's language	Many Massachusetts students are English learners; Spanish and Portuguese are common home languages.	Low with LLM; fallback templates in 2 languages.
No install, no account, runs client-side	Works on managed school Chromebooks and library PCs; nothing to approve or provision.	Already in the architecture.

Who benefits

Intro CS students in schools without a dedicated CS teacher, learners in library or after-school coding programs, English learners, and students who use screen readers. Scaling path: a teacher shares a lesson link; a library loads it on public machines; no server cost grows with users except optional explanations, which are cached.

One-sentence pitch

“CodeLens gives every beginner a patient tutor: it runs their real Python, shows each step, asks them to predict what happens next, and explains it in their own language, with nothing to install and no account.”

5 Architecture

[CHANGED] Recommended pathway: Vite + React + TypeScript static site → dedicated Web Worker running Pyodide → pure-Python tracer module → serialized TraceResult JSON → immutable timeline viewer. One serverless function, `/api/explain`, holds the LLM key and is optional. Deploy the static site and function together (Vercel, Netlify, or AWS Amplify, whichever the integrator already knows). Deploy a preview by 2:30 PM, not at the end.

Why drop FastAPI: it adds a second deploy target and a running server for one endpoint. Keep it only if the team already has hosting ready. Never run student code on any server.

Key design choice: the tracer is a plain Python file (`tracer.py`) loaded into Pyodide at runtime and imported directly by pytest in CPython. One implementation, testable at full speed on a laptop.

Folder layout

```
codelens/
  .kiro/                      <- commit this; required by the rules
    steering/  product.md  tech.md  structure.md  pedagogy.md
    specs/     trace-engine/  timeline-ui/  explain-service/   (requirements, design, tasks)
    hooks/     test-on-save, contract-guard, usage-log
  frontend/src/components/{Editor,Timeline,Variables,Arrays,Explanation,Predict,Narrator}.tsx
  frontend/src/worker/trace.worker.ts      frontend/src/py/tracer.py
  frontend/src/types/trace.ts              frontend/src/lessons.ts
  frontend/src/explain/templates.ts        (deterministic explanations, en + es)
  api/explain.ts                            (serverless function)
  shared/fixtures/*.json                    (precomputed traces for the 3 lessons)
  tests/test_tracer.py  tests/test_tracer_properties.py   (pytest + Hypothesis)
  README.md  KIRO_USAGE.md  LICENSE
```

Shared contract: freeze before parallel work

Every owner and every agent works against these types. Put them in a steering file so Kiro treats them as fixed.

```
TraceStep = {
  index: number; line: number; event: "call" | "line" | "return" | "exception";
  functionName: string; frameId: string; depth: number;
  locals: Record<string, JsonValue>;  stdout: string;   // cumulative
  returnValue?: JsonValue; error?: { type: string; message: string };
}
TraceResult = { steps: TraceStep[]; status: "ok" | "error" | "limit" | "timeout";
                message?: string; truncated: boolean; codeHash: string; }
```

[CHANGED] Changes from v1: `call` events for clearer function entry, structured `error` so the error explainer can switch on the type, a separate `timeout` status, and `codeHash` for caching explanations and share links.

6 Tracing engine

Use `sys.settrace` for call, line, return and exception events. Line events fire *before* the line runs, so the UI says “Next line to run”, shows locals captured at that instant, and marks changes by diffing against the previous snapshot.

Pipeline

- Compile the student's source with the synthetic filename `codelens_user.py`; surface SyntaxError as a terminal error step before tracing.
- Record only frames whose filename matches; ignore tracer internals and libraries.
- Snapshot sanitized locals: primitives, and lists/dicts/tuples of primitives up to a size cap; anything else becomes a type-only placeholder. Never call arbitrary `repr`.
- Capture stdout by redirecting `sys.stdout` to a buffer with a byte cap.
- Return one TraceResult; stepping never re-executes.

Fixes to v1's limits [CHANGED]

- **The step cap is the main infinite-loop guard.** Every iteration of a Python loop fires line events, so a 300-step cap stops `while True: pass` immediately without killing the worker. The worker timeout is only a backstop for loops inside C code (for example a huge `sum(range(10**10))`).
- Raise the cap exception as a subclass of `BaseException`, not `Exception`, so student code with `except Exception` cannot swallow it.
- Replace `input()` with a stub that raises a friendly “input is not supported in this lesson” error (or reads from a preset input box if time allows).
- Terminating a worker means reloading Pyodide (seconds). After a timeout, start a fresh worker immediately in the background so the next Visualize is fast.
- Preload the worker as soon as the page opens; show a progress indicator; render the lesson's precomputed fixture trace instantly so the page is never blank.

Tracer core (sketch)

```
class StepLimit(BaseException): pass

def tracer(frame, event, arg):
    if frame.f_code.co_filename != "codelens_user.py":
        return tracer
    if event in ("call", "line", "return", "exception"):
        if len(steps) >= MAX_STEPS:
            raise StepLimit()
        steps.append(snapshot(frame, event, arg))
    return tracer

sys.settrace(tracer)
try:
    exec(compile(src, "codelens_user.py", "exec"), {"__name__": "__main__", "input": no_input})
except StepLimit:  status = "limit"
except Exception as e:  status = "error"; record_error(e)
finally:
    sys.settrace(None)
```

Edge cases

Case	Required behavior
Loop with repeated line numbers	Monotonic step index; line number is not unique.
Nested functions	Stable frame IDs; correct depth; per-frame locals.
Exception while running	Prior snapshots plus a terminal error step with type, message and line.
Syntax error	No trace; single error state pointing at the reported line.
Infinite Python loop	Step cap ends it as status “limit”; worker stays alive.
Long-running C-level call	Worker timeout (about 3 s) ends it as status “timeout”; fresh worker warms in background.
Huge structure or output	Truncate and set truncated = true; label truncation in the UI.
Non-JSON objects	Type-only placeholder such as <object of type Foo>.

7 Frontend

The single source of truth is `result.steps[currentIndex]`. Every panel derives its view from it; no duplicated variable state in React.

Component	Inputs	Behavior
LessonPicker	lessons.ts	Three lessons with plain labels and DLCS tags; loads code and fixture.
CodeEditor	code, activeLine	Highlighting; read-only while tracing; Visualize button; gutter arrow on next line.
ExecutionControls	index, count	Previous/Next, Home/End, arrow-key shortcuts, “Step 5 of 18”.
VariablePanel	current and previous locals	Names, values, before→after markers with text, not just color.
ArrayVisualizer	list-valued locals	Indexed cells; changed cells marked.
[NEW] PredictPrompt	current and next step	Asks for the next value of a changing variable; checks against the trace; never blocks stepping.

Component	Inputs	Behavior
[NEW] ErrorExplainer	terminal error step	Plain explanation of the exception and the line; LLM version if available.
ExplanationPanel	code, current, previous	Deterministic text immediately; LLM text replaces it when it arrives; language toggle.
[NEW] Narrator	current step	Visually hidden aria-live region that announces each step.
TraceStatus	status, message	Loading Python, running, ready, error, limit, timeout.

State machine

LoadingPython → Idle → Running → Ready → (Error | Limit | Timeout). Disable Visualize while running. Editing the code invalidates the current trace. Focus moves predictably; disabled controls are announced.

8 AI integration: explain evidence, do not invent it

[CHANGED] Deterministic explanations come first and are always on, generated from the diff between snapshots (for example “The loop gave n its next value, 1. Next, line 4 will add n to total.”). The LLM improves wording and translation; it is never the source of a value. Cache by codeHash + step index + language.

Endpoint contract

```
POST /api/explain
{ "codeSlice": "...", "level": "beginner", "language": "es",
  "current":  { "index": 4, "line": 4, "event": "line", "locals": {"total": 3, "n": 1} },
  "previous": { "index": 3, "line": 3, "event": "line", "locals": {"total": 3, "n": 3} } }
200 OK  { "explanation": "...", "question": "What will total be after line 4 runs?" }
```

Prompt constraints

“You are a patient programming tutor for beginners. Explain this trace snapshot in one or two short sentences in {language}. Use only values that appear in the supplied snapshots; never state a future value. The highlighted line has not run yet; say so. Use the program's variable names. If asked for anything other than explaining this program, decline briefly.”

- **[NEW]** Server-side check: if the reply contains a number that is not present in either snapshot or the code slice, discard it and show the deterministic text. Cheap and makes “grounded” a verifiable claim.
- Cap payload size, rate-limit per IP, 5-second timeout, no logging of source code by default.
- Disclose in the UI that explanations send the selected code snippet to an external model service.
- Pick the model provider and confirm a working key by 11:00 AM. If there is no key by 2:00 PM, ship deterministic explanations only (Tier 2).

9 Kiro-native development plan **[NEW]**

This section is what the Implementation criterion scores. The goal is not to use every feature; it is to use each one for a reason you can explain and show in the repo and video.

9.1 Steering (context Kiro always sees)

File	Contents
product.md	Users, civic goal, MVP scope, non-goals, the definition of done.

File	Contents
tech.md	Stack, commands (dev, test, build, deploy), the frozen TraceStep/TraceResult contract, “never run student code on a server”.
structure.md	Folder layout and which owner or agent may edit which paths.
pedagogy.md	Explanation tone, reading level, the “next line to run” rule, never invent values, accessibility rules.

9.2 Specs (one per feature)

Create three specs in the Kiro IDE: **trace-engine**, **timeline-ui**, **explain-service**. Have Kiro draft requirements with testable acceptance criteria, then review and edit them yourself before generating design and tasks. Tasks become the units of work handed to agents and the units of commits.

9.3 Hooks (automation you can demonstrate)

Hook	Trigger	Action
test-on-save	Saving files under py/ or tests/	Run pytest including property tests; report failures to the agent.
contract-guard	Editing types/trace.ts or tracer.py output shape	Regenerate fixtures, run TS typecheck, warn that the shared contract changed.
a11y-check	Saving a component file	Ask the agent to check labels, focus order and color-only signals against pedagogy.md.
usage-log	Agent finishes a task	Append a one-line entry to KIRO_USAGE.md: what was asked, which feature or surface, result.

Check the exact hook trigger options in your Kiro version and adapt; keep at least two hooks that visibly fire during the video.

9.4 MCP

Use a GitHub MCP server for issues and pull requests, and a browser automation MCP (for example Playwright) to run the keyboard-only smoke test and capture screenshots for the README. Mention in the write-up what each made possible.

9.5 Property-based tests on the tracer

Kiro can derive properties from spec requirements. The tracer is ideal for this because it is pure Python and has clear invariants:

- Final-state agreement: for generated small programs (integer assignments, for/while loops, if/else), the locals at the last step equal the namespace from running the same program untraced.
- Indices are contiguous from 0, and len(steps) never exceeds MAX_STEPS.
- Every step's line is within the program's line range; depth is never negative.
- Every step serializes to JSON under the byte cap; truncated is true whenever anything was cut.
- Determinism: tracing the same program twice yields identical results.

Record any bug a property test caught; it is exactly the evidence the write-up asks for.

9.6 Kiro surfaces

- **Kiro IDE:** write steering and specs, review generated designs, UI polish where you want to see changes live.
- **Kiro Crew:** run parallel build waves where independent spec tasks go to separate subagents with isolated context; use the activity view to watch them and keep screenshots for the write-up.

- **Kiro CLI:** quick one-off tasks and test runs from the terminal.
- Using more than one surface is explicitly rewarded; note in KIRO_USAGE.md how they differed.

9.7 Evidence log

Keep KIRO_USAGE.md growing all day (the usage-log hook helps). At 5:30 PM, turn it into the required write-up with sections that match the rules' questions: tools used, spec-driven approach vs vibe coding, hooks, steering, MCP, property-based testing, powers, and Crew.

10 Team and integration protocol

Owner	Primary tasks	Kiro focus	Handoff
A: execution	tracer.py, worker, limits, fixtures	trace-engine spec, property tests, test-on-save hook	Real TraceResult for all 3 lessons by 1:00 PM
B: shell + integrator	Layout, editor, state machine, deploy	Steering files, Crew waves, GitHub MCP, merges	Mocked end-to-end by 12:00; preview deploy by 2:30
C: visualization	Controls, variables, arrays, predict, narrator	timeline-ui spec, a11y hook, browser MCP smoke test	Renders fixtures with predict by 2:00 PM
D: explain + story	Templates, /api/explain, i18n, README, video	explain-service spec, usage-log hook, KIRO_USAGE.md	Explanations never block by 3:30; write-up draft by 5:30

- The integrator owns types/trace.ts, merges, the deploy, and the release cutoff.
- Each owner works on a short-lived branch; merge to main every 60–90 minutes. main is always deployable and auto-deploys.
- Never let two agents edit the same file in parallel. File ownership lives in structure.md.
- No contract changes without telling every owner; the contract-guard hook makes this visible.

11 Build schedule: Saturday, October 24

[CHANGED] Assumes building from about 10:00 AM with submission locking at 8:00 PM. Gates matter more than clock times.

Window	Work	Exit gate
10:00–10:45	Repo, .kiro steering, contract, fixture for lesson 1, three specs drafted and reviewed. First commit includes .kiro.	Specs approved; contract frozen
10:45–12:30	Crew wave 1 in parallel: tracer + property tests; UI shell + timeline on fixture; deterministic templates + explain stub.	Mocked end-to-end Visualize and stepping
12:30–1:00	Lunch. Integrator merges and reviews wave 1.	main runs locally
1:00–2:30	Wave 2: real Pyodide traces in UI, 3 lessons, editing, limits. Deploy preview.	Loop, branch, function pass on the deployed URL
2:30–4:00	Wave 3: predict-then-reveal, error explainer, narrator, LLM explanations, share link.	Explanations never block tracing
4:00–5:00	Hardening: full test matrix, browser MCP smoke test, language toggle if on track.	All key tests pass
5:00	Feature freeze.	Release candidate tagged

Window	Work	Exit gate
5:00–6:15	README, KIRO_USAGE write-up, description, screenshots; record the video.	Video cut under 3:00
6:15–7:00	Upload video publicly; make repo public; verify .kiro present; final deploy.	Public links work in a private browser window
7:00–7:45	Fill in submission (track: Civic Action), smoke test, submit.	Confirmation received

Hard cutoffs

By 2:00 PM, if real tracing is unstable, ship fixture traces (Tier 3). By 2:00 PM, no LLM key means deterministic only. By 4:00 PM, stop on recursion and language toggle. By 5:00 PM, no new features. By 7:00 PM, only submission work.

12 Test matrix and fallback tiers

Test	Expected result
for loop + accumulator	total progresses 0 → 3 → 4 → 6; repeated line steps appear; predict accepts 4.
if/else	Only the executed branch appears.
function call / return	Depth and return value correct, or hidden if unsupported.
zero-iteration loop	No fake iterations.
NameError / ZeroDivisionError	Terminal error step; explainer text; UI usable.
SyntaxError	Error state at the right line; no trace.
while True: pass	Status “limit” in under 2 s; worker alive.
Long C-level call	Status “timeout”; next Visualize works without page refresh.
Explain endpoint offline	Deterministic explanations everywhere.
LLM returns an invented number	Server check discards it; deterministic text shown.
Rapid Next/Previous	No desync between panels.
Keyboard and screen reader	Every action reachable; each step announced.
Property tests	All Hypothesis properties pass at default example counts.

Tier 1: editable code, real traces, predict, error explainer, LLM explanations. **Tier 2:** same with deterministic explanations only. **Tier 3:** precomputed traces for the three lessons with the full step UI, stated honestly in the demo.

13 Demo video script (under 3:00) [CHANGED]

Judges are not required to watch past three minutes and may judge from the video alone. Aim for 2:45.

Time	Show
0:00–0:20	The problem and who it is for: a beginner with no tutor, on a school Chromebook.
0:20–0:50	Pick the loop lesson, Visualize, step through; point at “next line to run” and array cells.
0:50–1:20	Predict-then-reveal; the grounded explanation; switch the explanation to Spanish.
1:20–1:45	Edit the code to cause an error; show the error explainer. Run an infinite loop; show the graceful limit.
1:45–2:25	How it was built with Kiro: steering and a spec, a hook firing, Crew running parallel agents, a property test that caught a bug.
2:25–2:45	Civic scale: no accounts, share link, runs client-side, DLCS-aligned lessons; honest limits and next steps.

14 Submission checklist (mapped to the rules)

Requirement	Status check
All team members added; max 4; one representative submits	Names on the SundAI project card
Built with Kiro	KIRO_USAGE.md and commit history show it
Track identified	Civic Action
Text description of features and functionality	Written from README intro
Video under 3 minutes, public on YouTube or Vimeo, shows the project working	Link opens in a logged-out browser
Public GitHub repo with all source, assets and setup instructions	Fresh clone + README steps works
/.kiro at repo root and not in .gitignore	Check on github.com, not locally
Write-up on how Kiro was used	Covers tools, specs, hooks, steering, MCP, PBT, powers, Crew
Working test link, free to use	Deployed URL in the submission
No third-party trademarks or copyrighted music in the video	Use no music or royalty-free audio
Open-source use complies with licenses	LICENSE file; Pyodide and libraries credited

15 Risks and credit budget

Risk	Mitigation
Kiro credits run out mid-afternoon	Use Auto or a Sonnet-class model for implementation waves; reserve the strongest model for specs, design review and hard bugs. Check the usage meter after wave 1 and scale back parallelism if needed.
Parallel agents produce conflicting edits	Disjoint file ownership per agent; agents do not run git; the integrator commits each completed task.
Contract drift between agents	Contract in steering, contract-guard hook, shared fixtures as the single example.

Risk	Mitigation
Venue Wi-Fi slows Pyodide	Preload, fixture-first render, record the video on a warm cache.
Late deployment failure	Preview deploy by 2:30 PM; main auto-deploys afterward.
Video processing delay	Upload by 7:00 PM; YouTube processing can take minutes.
Submission rejected for missing .kiro or private repo	Check both on github.com from a logged-out window before submitting.

Bottom line: a working, accurate trace viewer that asks students to predict and explains only what really happened, plus a clear, evidenced story of building it with Kiro, is the win condition. Ship that before any stretch feature.